

MBEDX

ESP32 Embedded Systems Workshop

From Registers to IoT — Learn. Build. Deploy.

2 Days • 10 Hours • 12 Modules • PlatformIO + ESP-IDF • 11:00 AM – 5:00 PM

DAY 2

Connectivity, IoT & The Challenge

millis() • Deep Sleep • IoT Web Server • Team Algorithm Sprint

4

Modules

75 min

Challenge

3

Problems

1st

Team Wins

Day 2 — Schedule at a Glance

Connectivity & IoT • 11:00 AM – 5:00 PM

11:00 – 11:15	Day 1 Recap & Warm-Up	<i>millis()</i> warm-up question, quiz answers
11:15 – 12:00	Module 9 — Advanced Peripherals	<i>millis()</i> non-blocking, Hall sensor, Touch pins
12:00 – 12:30	Module 10 — Power Management	<i>Deep Sleep, timer wakeup, GPIO wakeup, RTC memory</i>
12:30 – 13:00	Module 11 — RTC	<i>Internal Real Time Clock</i>
13:00 – 13:30	Module 12 — SPIFFS	<i>Internal File system inside flash memory</i>
13:30 – 14:45	Module 9 — IoT & Web Server	<i>Wi-Fi, HTTP, live dashboard, remote control</i>
14:00 – 14:45	Lunch Break 🍴	
14:45 – 15:00	Challenge Briefing & Rules	<i>Problem statements revealed at 15:00 sharp</i>
15:00 – 16:30	🏆 Team Challenge Sprint	<i>All 3 challenges — first to complete all 3 wins</i>
16:30 – 17:00	Results, Prizes & Closing	<i>Trophy ceremony, what's next, resource pack</i>

Advanced Peripherals & millis()

Non-Blocking Code • Hall Effect • Capacitive Touch

Non-Blocking Code with millis()

delay() kills your firmware — millis() sets it free

✗ delay() — CPU is DEAD

```
void loop() {  
  // Can't read sensors  
  // Can't serve web pages  
  // Can't handle buttons  
  digitalWrite(LED, HIGH);  
  delay(1000); // CPU DEAD here  
  digitalWrite(LED, LOW);  
  delay(1000);  
}
```

✓ millis() — CPU stays ALIVE

```
unsigned long prev = 0;  
bool state = false;  
void loop() {  
  if (millis() - prev >= 1000) {  
    prev = millis();  
    state = !state;  
    digitalWrite(LED, state);  
  }  
  // CPU free for anything!  
}
```

Hall Effect Sensor (Built-In!)

No wiring needed — built into ESP32 chip.
hallRead() → returns field strength.
Bring phone near chip — watch value change.

Capacitive Touch Pins T0–T9

Wire 15cm jumper to GPIO4 (T0) as antenna.
touchRead(T0) → ~60 idle, ~10 when touched.
No mechanical parts — touchless switch!

Power Management & Deep Sleep

10

10 μ A Sleep • Timer Wakeup • GPIO Wakeup • RTC Memory

Deep Sleep — Making IoT Devices Last

Active: 240mA vs Deep Sleep: 0.01mA = 1000× longer battery life

ESP32 Power Modes

Active ~240 mA

Full speed — Wi-Fi, CPU, peripherals

Modem Sleep ~20 mA

CPU runs, Wi-Fi off between TX

Light Sleep ~2 mA

CPU paused, RAM retained

Deep Sleep ~0.01 mA

Only RTC — 1000× battery savings ✓

Exercises

```
RTC_DATA_ATTR int bootCount = 0;
void setup() {
  bootCount++;
  Serial.printf("Boot #%d\n", bootCount);
  // ...read sensor, blink LED...
  esp_sleep_enable_timer_wakeup(
    10ULL * 1000000); // 10 seconds
  esp_deep_sleep_start();
}
```

Ex 8.1 — Timer Wakeup:

Blink LED, sleep 10s, wake, print boot count (RTC_DATA_ATTR survives sleep), repeat.

Ex 8.2 — GPIO Button Wakeup (EXT0):

Sleep until GPIO15 button pressed → wake → read DHT11 → print temp → sleep again. This is how smart door sensors work.

Real Math: Every 5min active — 3s @ 240mA + 297s @ 0.01mA = avg 2.4mA → 2000mAh battery lasts 34 days vs 8 hours without sleep.

DAY 2

MODULE 8

RTC — Real Time Clock

Timekeeping • Alarms • Timestamps



RTC — Real Time Clock

Track time using the ESP32's built-in RTC — no extra hardware needed, powered by the same chip

How the Internal RTC Works?

Clock Source RTC runs on a 150 kHz internal oscillator (or external 32.768 kHz crystal if fitted)

Survives Deep Sleep The RTC domain stays powered even in deep sleep (~6.5 μ A) — time keeps ticking

Does NOT survive power-off No battery backup — loses time if 3.3V is removed completely

Drift ~1–2 minutes per day — acceptable for logging; use NTP sync over Wi-Fi for precision

Resolution 1 second via `getLocalTime()`, microseconds via `esp_timer_get_time()`

Setting Time: Two Methods

Method	Use When
<code>configTime()</code> + NTP	Wi-Fi available — syncs from internet, most accurate
<code>struct tm</code> manual set	No Wi-Fi — hardcode date/time at upload

Ex 8.1 — Log DHT11 readings with timestamp: "2025/03/08 14:32:05 | Temp: 28.4°C | Hum: 62%"

Ex 8.2 — Trigger buzzer alarm at a specific hour:minute using `timeinfo.tm_hour` and `timeinfo.tm_min`

RTCLib Code — Set, Read & Alarm

```
// platformio.ini — no extra lib_deps needed!
#include <WiFi.h>
#include "time.h"

const char* ntpServer = "pool.ntp.org";
const long  gmtoffset = 19800; // IST = UTC+5:30 (seconds)
const int   daylightOffset = 0;

void setup() {
  Serial.begin(115200);
  WiFi.begin("SSID", "PASSWORD");
  while (WiFi.status() != WL_CONNECTED) delay(500);

  // Sync time from internet
  configTime(gmtoffset, daylightOffset, ntpServer);
}

void loop() {
  struct tm timeinfo;
  if (!getLocalTime(&timeinfo)) {
    Serial.println("Failed to get time");
    return;
  }
  // Print formatted date & time
  Serial.printf("%04d/%02d/%02d %02d:%02d:%02d\n",
    timeinfo.tm_year + 1900,
    timeinfo.tm_mon + 1,
    timeinfo.tm_mday,
    timeinfo.tm_hour,
    timeinfo.tm_min,
    timeinfo.tm_sec);
  delay(1000);
}
```

DAY 2

MODULE 9

SPIFFS — File System Storage

Flash File System • Config Files • Logging

SPIFFS — Serial Peripheral Interface Flash File System

Store HTML, config files, images, and data logs directly on the ESP32's own flash chip

What is SPIFFS?

Flash Partition

Uses unused portion of the 4 MB Flash chip alongside your code — typically 1–3 MB available.

Flat File System

No folders — files live at paths like /index.html
/config.json /data.csv /logo.png

Use Cases

Serve HTML/CSS/JS from ESP32 as web server, store sensor logs, save config settings as JSON.

Storage Options — Quick Comparison

Type	Size	Access	Best For
EEPROM	~512 B	Bytes	Settings, flags
SPIFFS	1–3 MB	Files	HTML, logs, JSON
SD Card	GB+	Files	Large data logging

Ex 9.1 — Write + read a config.txt file

Ex 9.2 — Log DHT11 readings with timestamps to /log.csv

SPIFFS API — Read, Write & List Files

```
#include <SPIFFS.h>
SPIFFS.begin(true); // true = format if needed

// — WRITE a file —
File f = SPIFFS.open("/config.txt", "w");
f.println("threshold=35");
f.println("name=ESP32_Node1");
f.close();

// — READ a file —
File f2 = SPIFFS.open("/config.txt", "r");
while (f2.available()) {
  Serial.println(f2.readStringUntil('\n'));
}
f2.close();

// — LIST all files —
File root = SPIFFS.open("/");
File entry = root.openNextFile();
while (entry) {
  Serial.printf("%s %d bytes\n",
    entry.name(), entry.size());
  entry = root.openNextFile();
}
```

DAY 2

MODULE 9

IoT & Web Server

Wi-Fi Modes • HTTP Server • Live Dashboard • Remote Control

ESP32 Wi-Fi & HTTP Server

STA Mode • AP Mode • HTTP Request/Response • Remote Control

Wi-Fi Modes

STA (Station)

Connects TO existing Wi-Fi. Gets IP from router.

AP (Access Point)

Creates its OWN hotspot. Phone connects to ESP32 directly.

STA+AP (Both)

Connected to router AND hosting hotspot simultaneously.

HTTP Request / Response Cycle

- 1 Browser sends: GET /sensor HTTP/1.1
- 2 ESP32 receives: `server.handleClient()`
- 3 ESP32 responds: HTML with sensor values
- 4 Browser renders: Your dashboard appears!

⚠ ADC2 + Wi-Fi Conflict: GPIO 0,2,4,12-15,25-27 unreliable when Wi-Fi is ON — always use ADC1 (GPIO 32-39).

```
WiFi.begin("WorkshopSSID", "password");
while (WiFi.status() != WL_CONNECTED) { delay(500); }
Serial.println(WiFi.localIP()); // e.g. 192.168.1.105 – open in phone browser!
server.on("/", handleRoot); server.on("/led/on", handleLEDon); server.begin();
```

Building Your IoT Dashboard

From 'Hello World' to live sensor control — step by step

Ex 9.1 Connect to Wi-Fi & Get IP

Connect to workshop Wi-Fi. Print IP. Open `http://[IP]` in phone browser.

Ex 9.2 Serve a Web Page

Return `<h1>Hello from ESP32!</h1>`. Style with inline CSS. View in browser.

Ex 9.3 Live Sensor Dashboard

Auto-refresh every 3s. Show Temp, Humidity, Light%, Motion with color coding.

Ex 9.4 Remote Hardware Control

Add [LED ON] [LED OFF] [Buzz] buttons. Click in browser → hardware responds.



Ex 9.5 — Complete IoT Dashboard

- | | | |
|--|-----------------------------------|-----------------------------|
| 1 All sensor readings live | 2 Auto-refresh every 3s | 3 LED ON/OFF + Buzz buttons |
| 4 Uptime counter via <code>millis()</code> | 5 NORMAL / ALERT status indicator | |



LUNCH

45 minutes • Back at 14:45 sharp

 **THE CHALLENGE BEGINS AT 15:00**

Review your code • Discuss strategy • Refresh the concepts

No AI tools during the challenge — documentation only!



THE TEAM CHALLENGE

Algorithm & Logic Sprint • 33 Teams • 75 Minutes

FORMAT: Complete all 3 challenges in sequence (C1 → C2 → C3)

First team to have all 3 judge-verified wins. Ranked by: challenges completed → time of last judge stamp.

✓ **Internet docs ALLOWED**

Datasheets, Arduino reference, Stack Overflow — use them!

⊘ **No Generative AI**

ChatGPT, Copilot, Gemini banned — violation = disqualification

🔧 **Kit components only**

No external libraries beyond workshop-used ones

🗋️ **Every member must explain**

Judge will ask any member about any part of the code

⚖️ **Judge's call is final**

Demo must work on demand — not just once in the past

❑ **Problems revealed at 15:00 — CLOCK STARTS**

CHALLENGE 1

The Watchdog

State Machine + Interrupt Logic • Est. 12–18 min for fast teams

3-State System — Must behave EXACTLY as described:			
State	Trigger Condition	LED Behavior	Buzzer
● IDLE	System start / after button reset	Slow blink 1s ON / 1s OFF	Silent
● WARNING	IR detects object	Fast blink 200ms ON / 200ms OFF	Single beep every 2s
● ALARM	IR detected > 5 continuous seconds	Solid ON — no blink	Rapid beep 100ms/100ms

Additional Rules

- All timing via millis() — NO delay() anywhere
- Button = RESET interrupt (any state → IDLE)
- Serial prints every transition: 'STATE: WARNING→ALARM | T=12340ms'
- Object removed in WARNING before 5s → back to IDLE

Judge Verification

- Trigger IR → verify WARNING blink rate
- Hold IR 6s → verify ALARM escalation
- Press button → verify IDLE reset
- Check Serial for correct transition logs
- Q: Draw the state machine diagram now

CHALLENGE 2

The Vault

Sequential Lock Algorithm • Pattern Recognition • Timeout Logic • Est. 18–25 min

Unlock Sequence — Must match EXACTLY:

- 1

Step 1 — LDR

Cover LDR — hold for 1+ second (light < threshold)
- 2

Step 2 — IR

IR detects object — within 3s of Step 1 completing
- 3

Step 3 — Button

Press button — within 3s of Step 2 completing

System Responses:

- Correct step:

LED blinks once (100ms) — acknowledgement
- Step timeout:

Descending 3-tone alert → reset to Step 1
- Wrong order:

Double-beep → immediate reset to Step 1
- Full sequence ✓:

LED solid + ascending melody + 'VAULT UNLOCKED | Xms'

Must Also Track:

- Attempt count (total tries)
- Success count (unlocks)
- Time of each step via millis()
- 3s window accurate to ±100ms

Judge Verification:

- Deliberate wrong sequence → must reject
- Let step timeout → alert + reset
- Correct full sequence → must unlock
- Judge times 3s window with phone
- Q: 'What tracks the current step?'

CHALLENGE 3

The Reactor

Sensor Fusion • Scoring Algorithm • Proportional Response • Est. 20–30 min

Threat Score Algorithm — Implement EXACTLY:

```
THREAT_SCORE = 0

IF temp > 35°C      → SCORE += 40
IF temp 30–35°C    → SCORE += 20

IF light < 20%      → SCORE += 30
IF light 20–40%    → SCORE += 15

IF IR detects motion → SCORE += 30
RANGE: 0 – 100
```

Additional Rules:

- Score recalculated every 500ms using millis()
- Serial: 'SCORE: 70 | CRITICAL – HIGH_TEMP, DARK, MOTION'
- Boundary crossings: ascending tones up, descending down
- No delay() anywhere in the code

Response by Score:

0 – 30	SAFE	Green LED solid	Silent
31 – 60	CAUTION	Green LED fast blink	Beep every 3s
61 – 100	CRITICAL	All LEDs rapid blink	Continuous beep

Judge Verification:

- Cover LDR → score changes
- Breathe on DHT11 → temp rises → score changes
- Trigger all 3 → must hit CRITICAL
- Judge asks team to predict score before each change
- Q: 'How would you add a 4th sensor?'

Scoring, Rankings & Prizes

First team verified on all 3 challenges wins 🏆



1st Place

First team verified on all 3 challenges



2nd Place

Second team verified on all 3



3rd Place

Third team verified on all 3

Tiebreaker: Both teams finish simultaneously? Judge asks one technical question — better answer wins.

You're Now an Embedded Engineer.

Here's what you built in 10 hours — and where to go next

✓ GPIO & Arduino

pinMode, digitalWrite, digitalRead, interrupts

✓ Bare Metal C

Register addresses, TRM navigation, volatile

✓ Analog (ADC)

12-bit sensing, voltage dividers, thresholds

✓ PWM

LEDC API, LED brightness, tone generation

✓ Protocols

DHT11 single-wire, library-based sensor comms

✓ Power Management

Deep sleep, 1000× battery life, RTC memory

What's Next on Your Embedded Journey:

FreeRTOS
Multitasking

MQTT
Cloud IoT

ESP-IDF
Full Framework

PCB Design
KiCad

OTA Updates
Production

TLS/Security
Encryption

📁 Resource Pack (shared now):

GitHub repo (all exercise code) • ESP32 TRM PDF • PlatformIO quickstart guide • Component datasheets